Workload Monitoring

> **Series:** K8S | **Notebook:** 5 of 12 | **Created:** January 2026 | **Last Updated:** 01/30/2026

Application-Level Observability in Kubernetes

Workload monitoring focuses on the application layer: deployments, pods, containers, and the services they provide. This notebook covers monitoring Kubernetes workloads from deployment health to service performance.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
DynaKube	cloudNativeFullStack or applicationMonitoring
Permissions	`metrics.read`, `entities.read`, `logs.read`
Knowledge	K8S-01 Fundamentals, K8S-04 Cluster Monitoring

1. Workload Types and Monitoring

Kubernetes Workload Controllers

Workload Type	Use Case	Monitoring Focus
Deployment	Stateless apps	Replica availability, rollout status
StatefulSet	Stateful apps (DBs)	Pod identity, persistent storage
DaemonSet	Node-level agents	Coverage across all nodes
Job	Batch processing	Completion rate, duration
CronJob	Scheduled tasks	Execution timing, failures

Dynatrace Workload Entities

Entity Type	Kubernetes Resource	Key Attributes
`CLOUD_APPLICATION`	Deployment, StatefulSet	Replicas, strategy
`CLOUD_APPLICATION_NAMESPACE`	Namespace	Labels, quotas
`PROCESS_GROUP_INSTANCE`	Pod/Container	Resources, image
`SERVICE`	Service (detected)	Endpoints, traffic

```
// List all workloads (deployments, statefulsets)
fetch dt.entity.cloud_application
| fields entity.name, tags
| sort entity.name asc
| limit 50
```

2. Deployment Health

Deployment Status Indicators

Metric	Description	Healthy State
Available Replicas	Pods passing readiness	Equals desired
Ready Replicas	Pods passing all probes	Equals desired
Updated Replicas	Pods with latest spec	Equals desired
Collision Count	Hash collisions	Zero

Rollout Monitoring

During deployments, monitor:
- Old ReplicaSet scaling down
- New ReplicaSet scaling up
- Pod readiness during transition

```
// Deployment events - track rollouts
fetch logs
| filter matchesPhrase(content, "deployment") and (matchesPhrase(content,
"scaled") or matchesPhrase(content, "updated") or matchesPhrase(content,
"rollout"))
| fields timestamp, content
| sort timestamp desc
| limit 30
```

```
// Pod restart counts - identify unstable workloads
fetch logs
| filter matchesPhrase(content, "restarted") or matchesPhrase(content,
"BackOff")
| fields timestamp, content
| sort timestamp desc
| limit 30
```

3. Pod and Container Metrics

Container Resource Metrics

Metric	Description	Alert Threshold
CPU Usage %	Usage vs. limit	>85% sustained
Memory Usage %	Usage vs. limit	>90%
CPU Throttled	Time throttled	>10% of time
Memory Working Set	Active memory	Approaching limit

Pod Lifecycle States

State	Description	Action
Pending	Waiting to schedule	Check resources, node selectors
Running	At least one container running	Normal
Succeeded	All containers exited 0	Job completed
Failed	Containers exited non-zero	Check logs
Unknown	Node communication lost	Check node health

```
// Container CPU usage - find high consumers
fetch dt.metrics
| filter metric.key == "dt.containers.cpu.usage_percent"
| summarize avgCpuUsage = avg(value), by:{dt.entity.container_group_instance}
| sort avgCpuUsage desc
| limit 15
```

```
// Container memory usage approaching limits
fetch dt.metrics
| filter metric.key == "dt.containers.memory.usage_percent"
| summarize avgMemUsage = avg(value), by:{dt.entity.container_group_instance}
| filter avgMemUsage &gt; 75
| sort avgMemUsage desc
| limit 15
```

```
// Container CPU throttling - performance impact
fetch dt.metrics
| filter metric.key == "dt.containers.cpu.throttled_time"
| summarize avgThrottled = avg(value), by:
{dt.entity.container_group_instance}
| filter avgThrottled &gt; 0
| sort avgThrottled desc
| limit 15
```

4. Service Performance

Service-Level Metrics

Metric	Description	SLO Example
Response Time	P50, P90, P99 latency	P99 < 500ms
Error Rate	Failed requests %	< 0.1%
Throughput	Requests per second	Based on capacity
Availability	Successful responses	> 99.9%

Golden Signals for Services

Signal	What to Monitor	DQL Approach
Latency	Response time distribution	Timeseries with percentiles
Traffic	Request rate	Count over time
Errors	Error rate, types	Filter by status code
Saturation	Resource utilization	CPU, memory, connections

```
// Service response time (spans)
fetch spans
| filter span.kind == "server"
| summarize
    p50 = percentile(duration, 50),
    p90 = percentile(duration, 90),
    p99 = percentile(duration, 99),
    by:{dt.entity.service}
| sort p99 desc
| limit 15
```

```
// Service error rates
fetch spans
| filter span.kind == "server"
| summarize
    total = count(),
    errors = countIf(otel.status_code == "ERROR"),
    by:{dt.entity.service}
| fieldsAdd errorRate = 100.0 * toDouble(errors) / toDouble(total)
| filter errors &gt; 0
| sort errorRate desc
| limit 15
```

```
// Service throughput
fetch spans
| filter span.kind == "server"
| summarize requestCount = count(), by:{dt.entity.service}
| sort requestCount desc
| limit 10
```

5. Distributed Tracing in K8s

Trace Context in Kubernetes

Distributed tracing in Kubernetes follows requests across:
- Ingress → Services
- Service → Service (east-west traffic)
- Service → External (databases, APIs)

Trace Attributes in K8s

Attribute	Source	Use Case
`k8s.namespace.name`	OneAgent	Filter by namespace
`k8s.deployment.name`	OneAgent	Identify workload
`k8s.pod.name`	OneAgent	Instance-level analysis
`k8s.container.name`	OneAgent	Container identification

```
// Traces by namespace
fetch spans
| filter isNotNull(k8s.namespace.name)
| summarize requestCount = count(), avgDuration = avg(duration), by:
{k8s.namespace.name}
| sort requestCount desc
| limit 15
```

```
// Slow traces by workload
fetch spans
| filter span.kind == "server" and duration &gt; 1000000000  // &gt; 1 second
| fields timestamp, trace.id, k8s.namespace.name, k8s.deployment.name,
duration
| sort duration desc
| limit 20
```

6. Application Logs

Container Log Collection

OneAgent collects container logs from:
- stdout/stderr streams
- Mounted log files (with configuration)

Log Attributes

Attribute	Description
`k8s.namespace.name`	Source namespace
`k8s.pod.name`	Source pod
`k8s.container.name`	Source container
`dt.entity.container_group_instance`	Entity relationship

```
// Application error logs by namespace
fetch logs
| filter loglevel == "ERROR" or loglevel == "SEVERE"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 15
```

```
// Recent application logs with context
fetch logs
| filter isNotNull(k8s.namespace.name)
| filter loglevel == "ERROR"
| fields timestamp, k8s.namespace.name, k8s.pod.name, content
| sort timestamp desc
| limit 30
```

7. Workload Anomalies

Common Anomaly Types

Anomaly	Indicators	Root Cause
Memory Leak	Steadily increasing memory	Application bug
CPU Spike	Sudden CPU increase	Traffic surge, infinite loop
Error Storm	Rapid error increase	Dependency failure
Latency Degradation	Increasing response time	Resource contention

Davis AI for Workloads

Dynatrace Davis AI automatically detects:
- Response time degradation
- Error rate increases
- Resource saturation
- Failure rate anomalies

```
// Error count by namespace (detecting increases)
fetch logs
| filter loglevel == "ERROR"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 10
```

```
// Memory usage by workload (detecting high consumers)
fetch dt.metrics
| filter metric.key == "dt.containers.memory.working_set_bytes"
| summarize avgMemoryBytes = avg(value), by:{dt.entity.cloud_application}
| sort avgMemoryBytes desc
| limit 10
```

Next Steps

With workload monitoring configured, proceed to:

Next Notebook	Topic
K8S-06: Namespace Organization	Boundaries and access control
K8S-07: Events and Logs	Kubernetes event analysis
K8S-08: DQL for Kubernetes	Advanced query patterns

Summary

In this notebook, you learned:

- Kubernetes workload types and their monitoring focus
- Deployment health indicators and rollout monitoring
- Pod and container resource metrics
- Service-level performance monitoring (golden signals)
- Distributed tracing with Kubernetes attributes
- Application log analysis by namespace and pod
- Workload anomaly detection patterns

References

- Kubernetes Workload Monitoring
- Container Monitoring
- Service Monitoring
